Replicating Avellaneda–Stoikov Optimal Market Making with Reinforcement Learning Policies: Pitfalls and Challenges

Lewis Wilcher

November 2026

Abstract

We solve the Avellaneda–Stoikov market making problem with exponential utility analytically, reducing it to a linear system of ordinary differential equations in inventory. We then try to recover the optimal quotes by reinforcement learning (REINFORCE) with six policy classes, from a four parameter model to a table of 851 numbers. All six fall short of the optimum, by between 0.0009 and 0.0103 in expected utility (0.04 to 0.45 in certainty equivalent), averaged over five seeds. A direct fit shows that the shortfall is not caused by the policy classes being unable to represent the optimal quotes. The evidence points to a weak gradient signal at the start of training, which gets worse as the number of parameters grows, and to a bias that the exploration noise puts into the learned spread. Removing that bias after training, or tying the ask and bid weights by symmetry, reduces the gap. We also list what we did not establish.

1 Introduction

A market maker posts a bid and an ask around a mid price and earns the spread when they are filled, but is left holding inventory that moves with the price. Avellaneda and Stoikov model this as a stochastic control problem with a closed form solution. This makes it a useful test bed for learning methods, because the answer a learner should find is known exactly.

This paper does two things. First, we give the analytical solution and what it says about the optimal quotes. Second, we try to replicate it with learned policies of different sizes and structures, and we study why the learned policies do not reach the optimum. The second part is the main subject. A learner that gets close to a known optimum is only mildly interesting. What is useful is understanding where it loses utility and why.

All learning code is written from scratch in pure Python, and all results are averaged over five random seeds unless stated otherwise.

2 Analytical Solution

2.1 Setup

The mid price follows $dS_t = \sigma\, dW_t$. The agent quotes an ask at $S_t + \delta_t^a$ and a bid at $S_t - \delta_t^b$. An ask is filled at rate $\lambda(\delta^a) = Ae^{-k\delta^a}$ and a bid at rate $\lambda(\delta^b) = Ae^{-k\delta^b}$. Inventory is q_t, so $dq_t = dN_t^b - dN_t^a$, and cash X_t changes by $S_t + \delta^a$ at an ask fill and by $-(S_t - \delta^b)$ at a bid fill. The agent maximises

$$\mathbb{E}\left[-\exp\big(-\gamma(X_T + q_T S_T)\big)\right]. \tag{1}$$

We use $\sigma = 2$, $A = 10$, $k = 0.5$, $\gamma = 0.2$ and $T = 1$ throughout.

2.2 Solution

Write the value function as $u(t, x, s, q)$. The Hamilton–Jacobi–Bellman equation is

$$u_t + \tfrac{1}{2}\sigma^2 u_{ss} + \max_{\delta^a} \lambda(\delta^a)\big[u(x+s+\delta^a, s, q-1) - u\big] + \max_{\delta^b} \lambda(\delta^b)\big[u(x-s+\delta^b, s, q+1) - u\big] = 0, \quad (2)$$

with $u(T, x, s, q) = -e^{-\gamma(x+qs)}$. The ansatz

$$u(t, x, s, q) = -\exp\big(-\gamma(x + qs + \theta(t, q))\big), \qquad \theta(T, q) = 0, \quad (3)$$

removes x and s from the problem. The first order conditions for the two maximisations give

$$\delta^{a*} = c + \theta(t, q) - \theta(t, q-1), \qquad \delta^{b*} = c + \theta(t, q) - \theta(t, q+1), \qquad c = \frac{1}{\gamma}\ln\Big(1 + \frac{\gamma}{k}\Big). \quad (4)$$

Substituting back and setting $v(t, q) = e^{k\theta(t,q)}$ gives a linear equation in v. In time to maturity $\tau = T - t$ it reads

$$\frac{\partial v(q)}{\partial \tau} = -\alpha q^2 v(q) + \eta\big[v(q-1) + v(q+1)\big], \qquad v(0, q) = 1, \quad (5)$$

with

$$\alpha = \tfrac{1}{2}k\gamma\sigma^2, \qquad \eta = \frac{kA}{k+\gamma}\Big(1 + \frac{\gamma}{k}\Big)^{-k/\gamma}. \quad (6)$$

Restricting inventory to $|q| \le Q$, this is $\dot v = Bv$ with a tridiagonal matrix B, where $B_{qq} = -\alpha q^2$ and $B_{q,q\pm1} = \eta$, so $v(\tau) = \exp(\tau B)\mathbf{1}$. The optimal quotes are

$$\delta^{a*}(\tau, q) = c + \frac{1}{k}\ln\frac{v(\tau, q)}{v(\tau, q-1)}, \qquad \delta^{b*}(\tau, q) = c + \frac{1}{k}\ln\frac{v(\tau, q)}{v(\tau, q+1)}. \quad (7)$$

For our parameters $c = 1.682$, $\alpha = 0.2$ and $\eta \approx 3.08$. Because $B_{q,q}$ is symmetric in q and the coupling is the same on both sides, the bid is the mirror image of the ask, $\delta^{b*}(\tau, q) = \delta^{a*}(\tau, -q)$.

2.3 What the optimal quotes do

At $\tau = 0$ both quotes equal c. Expanding to first order in τ gives

$$\delta^{a*} \approx c + \big(\tfrac{1}{2} - q\big)\gamma\sigma^2\tau, \qquad \delta^{b*} \approx c + \big(\tfrac{1}{2} + q\big)\gamma\sigma^2\tau. \quad (8)$$

So with more time left, both quotes widen (the $\frac{1}{2}$ term), because the agent is exposed to price risk for longer. A positive inventory then pulls the ask closer to the mid price and pushes the bid away, which makes a sale more likely and a further purchase less likely. A negative inventory does the opposite. The size of the adjustment grows with the remaining time, with inventory, and with $\gamma\sigma^2$.

The expansion is accurate only for small τ and $|q|$. Table 1 shows exact asks. At $\tau = 0.1$, $q = 0$ the expansion gives 1.722, which matches, but at $\tau = 1$, $q = 0$ it gives 2.082 against an exact 1.924.

For large inventory and long horizons the exact quote becomes negative. For example the optimal ask is -1.23 at $\tau = 1$, $q = 7$. A negative quote means selling below the mid price. In our simulation every quote, including the optimal one, is clipped at zero before it reaches the market. These states are rarely visited: 1.7% of the optimal agent's decisions have $|q| \ge 4$. We did not measure separately how much of any gap comes from them.

	$q=-2$	$q=0$	$q=2$
$\tau=0.1$	1.881	1.722	1.563
$\tau=0.5$	2.523	1.851	1.177
$\tau=1.0$	2.874	1.924	0.962

Table 1: Optimal ask $\delta^{a*}(\tau,q)$. The bid is $\delta^{b*}(\tau,q)=\delta^{a*}(\tau,-q)$.

3 Reinforcement Learning Setup

3.1 Environment

Time is discretised into $N=50$ steps of length $\Delta t=T/N$. At each step the agent chooses an ask and a bid distance. Each side fills independently with probability $1-e^{-\lambda(\delta)\Delta t}$, which is the exact probability of at least one Poisson arrival in the step. A fill changes inventory by one unit and cash by the quoted price. The price then moves by $\sigma\sqrt{\Delta t}\,Z$ with $Z\sim\mathcal{N}(0,1)$. The only reward is terminal, $R=-\exp(-\gamma W_T)$ with $W_T=X_T+q_TS_T$, so there is no intermediate feedback.

3.2 Policy

The policy outputs a mean ask $\mu^a(\tau,q)$ and mean bid $\mu^b(\tau,q)$. The sampled quotes are

$$\delta^a=\mu^a+\sigma_\pi\varepsilon^a,\qquad \delta^b=\mu^b+\sigma_\pi\varepsilon^b,\qquad \varepsilon\sim\mathcal{N}(0,1), \tag{9}$$

with exploration noise $\sigma_\pi=0.5$, and are clipped at zero before the market sees them. The inventory used to look up the policy is clamped to $|q|\le 7$.

3.3 Objective and gradient

We maximise $J(\vartheta)=\mathbb{E}[R]$ over the policy parameters ϑ. The REINFORCE estimator is

$$\nabla_\vartheta J=\mathbb{E}\Big[(R-b)\sum_{n=0}^{N-1}\frac{\varepsilon_n^a\nabla_\vartheta\mu_n^a+\varepsilon_n^b\nabla_\vartheta\mu_n^b}{\sigma_\pi}\Big], \tag{10}$$

where b is a baseline. We use the mean reward of the batch as b and divide the advantages $R-b$ by their batch standard deviation. Each batch has 500 episodes. For every step and inventory level we first add up the advantage-weighted noise into a table $W[n][q]$, and then back-propagate once per batch through the policy. This gives the same gradient as back-propagating each decision separately, at a much lower cost.

3.4 Update

Parameters are updated with Adam. The learning rate decays linearly from its initial value to 10% of it over training. The parameters used for evaluation are the average over the last 30% of iterations.

3.5 Evaluation

Policies are evaluated without exploration noise, so the quotes are the means μ, clipped at zero. Every policy, and the optimal reference, is run on the same 100,000 simulated paths (seed 123).

We report the paired gap $J^\star - J_\pi$, where $J^\star$ is the expected utility of the optimal quotes. We also report the certainty equivalent $\mathrm{CE} = -\ln(-J)/\gamma$, computed from the mean J. The optimal reference is computed with an inventory cap of 20 and Runge–Kutta steps of 0.005 for the ODE. The learners use a cap of $Q = 8$.

4 Learning Policies

We use six policy classes. They differ in how much structure from the analytical solution they build in. Table 2 summarises them. Learning rates and iteration counts were set per policy and are listed there.

Policy	Free parameters	Structure used	Learning rate	Iterations
Four parameter	4	full form of the solution	0.03	1500
Linear	8	none	0.03	2500
Neural network	42	none	0.01	1500
Tridiagonal matrix	50	tridiagonal B	0.01	1500
Dense matrix	290	matrix exponential only	0.01	1500
Value function table	851	quote formula only	0.003	1500

Table 2: The six policy classes.

Linear. The mean quotes are linear in the features $(1, q, \tau, q\tau)$, with separate weights for the ask and the bid, giving $4 + 4 = 8$ parameters. Both weight vectors start at $(1, 0, 0, 0)$. This is the simplest policy and mirrors the first order expansion of the optimal quotes.

Neural network. The inputs are $q/5$ and τ. There is one hidden layer of 8 tanh units and two outputs, the mean ask and the mean bid, giving $5 \cdot 8 + 2 = 42$ parameters.

Matrix policies. The next three policies all compute $v = \exp(\tau B)\mathbf{1}$ by fourth order Taylor steps of size Δt and set

$$\mu^a = c + s \ln \frac{v(q)}{v(q-1)}, \qquad \mu^b = c + s \ln \frac{v(q)}{v(q+1)}, \tag{11}$$

with c and s learned. The true values are $c = 1.682$ and $s = 1/k = 2$. The gradient is back-propagated through the Taylor steps by hand. The three differ in how B is parametrised, with 17 inventory states ($|q| \le 8$).

- *Dense matrix.* Every entry of the 17×17 matrix is free. The diagonal is learned directly and the off-diagonal entries as logs, so that they stay positive, starting at 0.05. The $q = 0$ diagonal entry is fixed at zero. With c and s this gives 290 free parameters. The true B is contained in this family only approximately, because a log parametrised entry can approach zero but not equal it.

- *Tridiagonal matrix.* Only the diagonal and the two neighbouring bands are free, with the bands as logs starting at 1. This contains the true B exactly, apart from the truncation of inventory, and has 50 free parameters.

- *Four parameter.* B is built from the formula $B_{qq} = -\alpha q^2$, $B_{q,q\pm1} = \eta$. The learned parameters are $\ln \alpha$, $\ln \eta$, c and s, starting at $\alpha = 0.05$, $\eta = 1$, $c = 1$, $s = 1$. This contains the optimum exactly.

Value function table. The policy stores $\theta(n, q)$ directly for every time step and inventory level, with c learned as well, and sets

$$\mu^a = c + \theta(n, q) - \theta(n, q-1), \qquad \mu^b = c + \theta(n, q) - \theta(n, q+1). \tag{12}$$

This has $50 \times 17 + 1 = 851$ parameters, starting from $\theta = 0$. It has no structure beyond the form of the quotes.

5 Results

5.1 Main comparison

Table 3 shows the results over five seeds, ordered from worst to best mean gap, with the optimal reference at the bottom.

Policy	Params	Mean J	CE	Gap (mean $\pm$ sd)
Value function table	851	-0.11929	10.631	0.01031 ± 0.00062
Dense matrix	290	-0.11500	10.814	0.00602 ± 0.00074
Tridiagonal matrix	50	-0.11223	10.936	0.00325 ± 0.00059
Linear	8	-0.11214	10.940	0.00316 ± 0.00101
Neural network	42	-0.11190	10.951	0.00291 ± 0.00139
Four parameter	4	-0.10986	11.043	0.00088 ± 0.00094
Optimal	—	-0.10898	11.083	—

Table 3: Expected utility J, certainty equivalent and paired gap to the optimum, over five seeds. Evaluation uses 100,000 shared paths without exploration noise.

Two things stand out. The table and the dense matrix are clearly the worst, and the four parameter policy is clearly the best. The tridiagonal, linear and neural network policies are not distinguishable: their means differ by less than their seed-to-seed standard deviations, so the order in which they appear in the table should not be read as a ranking. Earlier single-run comparisons suggested a ranking among these three, and five seeds showed it was not reliable.

5.2 Linear policy weights against the first order expansion

Table 4 compares the learned linear weights to the first order expansion of the optimal quotes.

The signs and rough sizes of the τ and $q\tau$ weights agree with the expansion, but the constants are too high, the ask constant by about 0.18 and the bid constant by about 0.39. The first order values are not the best linear approximation either. A weighted least squares fit of a linear policy to the exact optimal quotes (weights as in Section 6.1) gives $a = (1.866, -0.041, 0.237, -0.541)$ and $b = (1.866, 0.044, 0.235, 0.533)$. The learned weights are closer to this fit than to the expansion, in particular for the constant, which the fit also puts near 1.87. The two sides of the fit are mirror images, but the learned ask and bid constants differ (1.862 against 2.071). We checked that this is not a bug: the episode and gradient code is exactly mirror symmetric, and in 15 seeds the bid

Weight	Learned (mean ± sd)	First order
a_0 (ask, constant)	1.862 ± 0.067	1.682
a_1 (ask, q)	-0.032 ± 0.101	0
a_2 (ask, τ)	0.309 ± 0.124	0.400
a_3 (ask, $q\tau$)	-0.651 ± 0.194	-0.800
b_0 (bid, constant)	2.071 ± 0.075	1.682
b_1 (bid, q)	0.124 ± 0.085	0
b_2 (bid, τ)	0.273 ± 0.248	0.400
b_3 (bid, $q\tau$)	0.522 ± 0.169	0.800

Table 4: Learned linear weights over five seeds against the first order expansion.

constant was above the ask constant in 11, which is consistent with chance. We treat the difference as learning noise.

5.3 Four parameter policy

Although the four parameter policy reaches the lowest gap, it does not recover the parameters of the solution (Table 5).

Parameter	True	Learned (mean ± sd)
α	0.2	0.145 ± 0.012
η	3.08	0.73 ± 0.29
c	1.682	1.857 ± 0.079
s	2	2.014 ± 0.072

Table 5: Parameters learned by the four parameter policy over five seeds. Across seeds η ranges from 0.37 to 1.07.

The utility is close to optimal even though η is far from its true value. A least squares fit of the four numbers to the exact quotes also does not return the true values: it has a single minimum at $\alpha \approx 0.25$, $\eta \approx 2.49$, $s \approx 1.6$, $c \approx 1.682$, with a root mean square error of 0.0015. We did not establish why the fit and the truth differ.

6 Pitfalls and Challenges

6.1 Representation is not the limit

A first guess is that a simple policy such as the linear one cannot represent the non-linear optimal quotes. We tested this directly by fitting each of the linear and neural network families to the exact optimal quotes by weighted least squares, weighting each state by how often the optimal agent visits it. Using the fitted quotes as the policy, the best linear fit has a gap of 0.00003 (standard error 0.00006) and the 42 parameter network fit has no measurable gap. Both are far below the gaps in Table 3. The four parameter, tridiagonal and dense matrix policies contain the optimal quotes by construction (the dense one only approximately), so a representation test is not needed for them.

The shortfalls are therefore due to optimisation and not to the policy classes. They are also not due to overfitting: no regularisation is used, and every batch is made of fresh simulated episodes. The issue is the variance of the gradient estimate.

6.2 Weak gradient signal

The reward arrives only at the end of the episode and depends on three sources of randomness: the price path, the Poisson fills, and the exploration noise. We expect a parameter that influences only a few decisions to receive a small share of the signal, with the rest of the episode acting as noise for it.

We measured this at the start of training. For each parameter we computed its batch gradient on 40 independent batches and took the absolute value of the mean divided by the standard deviation across batches. Pure noise gives a value of about 0.1 for this quantity with 40 batches. Table 6 lists the values.

Parameter	Gradient signal to noise
c (matrix policies)	0.45 to 1.0
$\ln \alpha$	0.40
$\ln \eta$	0.175
Matrix entries (dense, tridiagonal)	0.1 to 0.15
Table cells $\theta(n, q)$	0.11
Linear weight on q	0.006
Linear weight on $q\tau$	0.03

Table 6: Gradient signal to noise at the start of training. A value near 0.1 is what pure noise gives.

Parameters that affect every decision, such as c, have the strongest signal. Parameters that affect few decisions, such as the table cells and individual matrix entries, are close to the noise floor. In the table policy each cell receives about 77 decisions per batch near $q = 0$ but only about 7 for $|q| \geq 4$. We read this as the reason that the largest policies do worst, but we did not confirm it with an intervention. The linear weights on q and $q\tau$ also have very low values at the starting point, and we did not investigate why. All of these were measured only at initialisation, and the signal may change during training.

6.3 Exploration noise biases the spread

The learner is trained on noisy quotes but evaluated on the noise-free mean. A noisy quote that is cheap fills more often than an expensive one, so for a given mean the average price obtained per fill is lower than at the noise-free quote. To compensate, the learner raises its mean quote. A small noise approximation of the first order condition suggests that the mean quote shifts up by about

$$\Delta \approx \left(k + \tfrac{1}{2}\gamma\right)\sigma_\pi^2 = 0.15 \tag{13}$$

for $\sigma_\pi = 0.5$. This is an approximation and we did not derive its error. Evaluated without noise, a constant upward shift of the optimal quotes costs 0.00035 in J for a shift of 0.15 and 0.00225 for a shift of 0.3. The learned c of the four parameter policy (Table 5) is 0.175 above its true value, and the learned bid constant (Table 4) is above the first order value, both in the direction this predicts. The learned ask constant is close to the best linear fit rather than above it, so the evidence is not uniform across parameters.

6.4 Interventions

We tested three changes aimed at these two effects, with five seeds each. The results are in Table 7.

Policy	Baseline	Retuned shift	Noise lowered	Tied
Linear	0.00316	0.00134	0.00435 ± 0.00296	0.00185 ± 0.00087
Four parameter	0.00088	0.00017	0.00052 ± 0.00053	—

Table 7: Mean gap to the optimum under each intervention over five seeds. The baseline standard deviations are 0.00101 (linear) and 0.00094 (four parameter), from Table 3.

Retuning the level without noise. After training, a single constant shift is added to the learned quotes. The shift is chosen from a grid from -0.40 to 0.20 in steps of 0.05 using 10,000 separate noise-free paths (seed 999), and the result is re-evaluated on the 100,000 evaluation paths. This reduces the gap from 0.00316 to 0.00134 for the linear policy and from 0.00088 to 0.00017 for the four parameter policy. The chosen shifts were mostly negative, between -0.30 and 0. After the shift the mean constant c was 1.747 for both policies, close to but still above 1.682. This supports the view that a large part of the remaining gap is a level error caused by the noise.

Lowering the noise during training. We lowered σ_π linearly from 0.5 to 0.1 over the second half of training. This helped the four parameter policy slightly (gap $0.00088 \to 0.00052$, mean learned c from 1.857 to 1.757) but made the linear policy worse ($0.00316 \to 0.00435$) with a large seed variation. We do not know why it hurt the linear policy.

Tying ask and bid weights. Since the optimal bid is the mirror of the ask, we tied the linear bid weights to the ask weights, leaving four free parameters. The gap fell from 0.00316 to 0.00185. The paired improvement across seeds is 0.00131 with standard error 0.00033. Using fewer parameters here helps, which is consistent with the gradient signal argument, but it also removes the ask and bid constant mismatch described above, so the two explanations cannot be separated.

6.5 Other pitfalls

- *Single runs mislead.* The seed-to-seed standard deviations are as large as the differences between several of the policies. Early conclusions drawn from single runs, including a ranking of the linear, tridiagonal and network policies, did not survive five seeds.
- *Parametrisation and initialisation matter.* Starting the tridiagonal couplings at 1 rather than 0.05 gave a gap of 0.0030 against 0.0064 in a single run. We did not repeat this over seeds or explain it.
- *Positivity.* The matrix couplings and α, η are parametrised as logs so that they stay positive. The cost is that a coupling that should be zero can only approach zero, which is why the dense family only approximately contains the optimum.
- *Negative optimal quotes.* The optimal quotes are negative at large inventory, but the market sees them clipped at zero. This affects rarely visited states and we did not quantify its effect.

- *Tuning on the evaluation paths.* Learning rates, iteration counts, initialisations and the noise schedule were chosen after looking at results on the same evaluation paths, so the reported gaps are somewhat optimistic for the final settings.

7 Conclusion and Future Work

The Avellaneda–Stoikov problem has a closed form solution through a linear ODE, and the optimal quotes are simple enough that a linear policy and a small network can fit them almost exactly. Despite this, none of the six learned policies reaches the optimum. The shortfall is an optimisation effect: a gradient signal that weakens as parameters are added, and an exploration noise that shifts the learned spread. Policies that carry more of the structure of the solution do better, but even the four parameter policy, which contains the optimum, does not recover the true parameters.